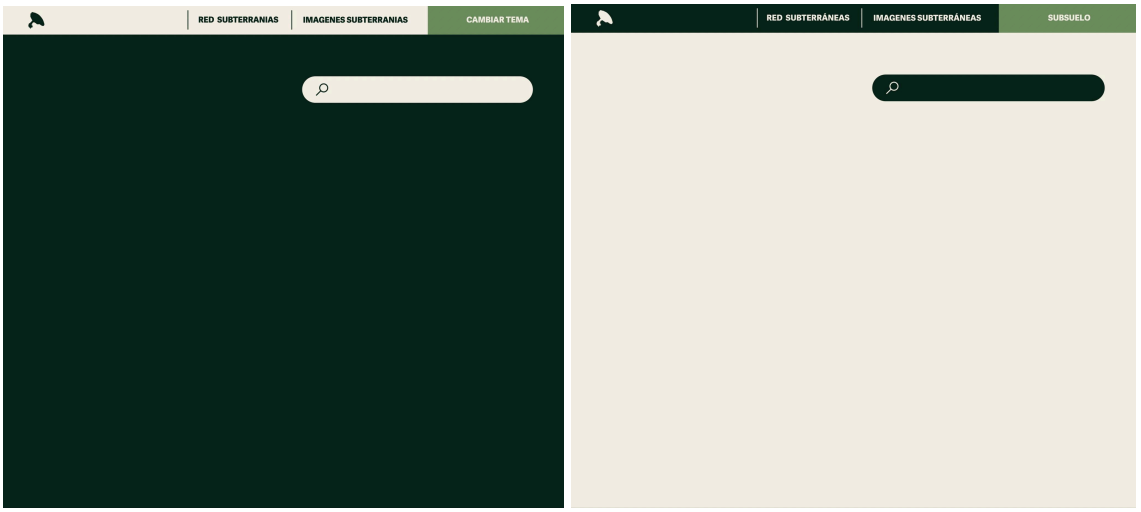
RED SUBTERRÁNEA

Informacion para la creacion del html que se encuentra dentro del heder llamada redes supteranis

medidas

Tamaño pantalla 1080x1920 ajustable a movil que mida 375x667

texto header adjunto imagenen de como debe de ir la composicion de esta pagina: modo light y modo dark 27puntos el tamaño adjuntar el honguito que es el boton que te regresa al home que estara en la carpeta imagenes- home- svg



boton de cambiar tema en la pantalla verde se llamara Bosque en la negra se llamara subsuelo-

Poner y activar el boton que de la lupita de busqueda palabras que se enlazaran a la telaraña de datos: datos con los id y las imagen que conectaran y el xcel convertido a un JSON

Heder esta en tipografia adele sans blak

- tres colores del modo claro con valores hexadecimales;



- tres colores del modo oscuro con valores hexadecimales;



- tres adjetivos que definan el tono visual;
  - Contemporáneo
  - Experimenta/Fotografico
  - Interactivo
  - Tecnologico
  -
- tres adjetivos que definan el tono verbal.
  - Clasico
  - Balanceado
  - Contraste

codigo a usar

codigode de prueba:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Telaraña de Información - Estilo Árbol</title>
    <!-- Cargamos D3.js desde una CDN estable -->
    <script src="https://cloudflare.com"></script>
    <style>
        body {
            margin: 0;
            overflow: hidden;
            background-color: #ffffff;
            font-family: 'Courier New', Courier, monospace; /* Tipografía idéntica a tu ejemplo */
            font-size: 11px;
        }

        #canvas-container {
            width: 100vw;
            height: 100vw;
```

```
    position: relative;
}

svg {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    z-index: 1;
}

/* Estilo de las líneas/ramas */
.link {
    stroke: #1a1a1a;
    stroke-width: 1px;
}

/* Contenedor de nodos HTML para poder renderizar las imágenes */
.nodes-container {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    z-index: 2;
    pointer-events: none; /* Deja que el SVG maneje el arrastre si es necesario */
}

/* Estructura de cada bloque de información */
.node-block {
    position: absolute;
    transform: translate(-50%, -50%);
    display: flex;
    flex-direction: column;
    align-items: flex-start;
    pointer-events: auto;
    cursor: move;
}

.node-label {
    background: rgba(255, 255, 255, 0.8);
    padding: 1px 3px;
    margin-bottom: 2px;
    white-space: nowrap;
}

.node-img {
```

```
            width: 60px;  /* Ajusta el tamaño de los trozos de imagen */
            height: 60px;
            object-fit: cover;
            border: 1px solid #1a1a1a;
            box-shadow: 2px 2px 5px rgba(0,0,0,0.1);
        }

        /* Punto de anclaje (el círculo negro/rojo de la esquina o centro) */
        .node-dot {
            width: 6px;
            height: 6px;
            background-color: #000000;
            border-radius: 50%;
            position: absolute;
            top: 0;
            left: 0;
            transform: translate(-50%, -50%);
        }

        /* Variante roja como en el lado derecho de tu ejemplo */
        .node-dot.red {
            background-color: #ff3333;
            width: 8px;
            height: 8px;
        }
    </style>
</head>
<body>

    <div id="canvas-container">
        <!-- El SVG dibujará los hilos/líneas de la telaraña -->
        <svg id="network-svg"></svg>
        <!-- Este div contendrá las fotos y textos alineados con los hilos -->
        <div id="html-nodes" class="nodes-container"></div>
    </div>

    <script>
        // 1. Estructura de Datos (Nodos y sus Relaciones)
        // Puedes cambiar las imágenes de ejemplo por las tuyas locales o URLs reales
        const dataset = {
            nodes: [
                { id: "trozos_90", label: "trozos_90", img: "https://picsum.photos", type: "black" },
                { id: "trozos_213", label: "trozos_213", img: "https://picsum.photos", type: "black" },
                { id: "trozos_182", label: "trozos_182", img: "https://picsum.photos", type: "black" },
                { id: "trozos_91", label: "trozos_91", img: "https://picsum.photos", type: "black" },
                { id: "trozos_100", label: "trozos_100", img: "https://picsum.photos", type: "red" },
                { id: "trozos_173", label: "trozos_173", img: "https://picsum.photos", type: "black" },
                { id: "trozos_66", label: "trozos_66", img: "https://picsum.photos", type: "black" },
```

```
            { id: "trozos_82", label: "trozos_82", img: "https://picsum.photos", type: "black" },
            { id: "trozos_46", label: "trozos_46", img: "https://picsum.photos", type: "red" },
            { id: "trozos_132", label: "trozos_132", img: "https://picsum.photos", type: "black" },
            { id: "trozos_62", label: "trozos_62", img: "https://picsum.photos", type: "black" },
            { id: "trozos_151", label: "trozos_151", img: "https://picsum.photos", type: "black" },
            { id: "trozos_211", label: "trozos_211", img: "https://picsum.photos", type: "black" },
            { id: "trozos_210", label: "trozos_210", img: "https://picsum.photos", type: "black" }
        ],
        links: [
            { source: "trozos_90", target: "trozos_100" },
            { source: "trozos_213", target: "trozos_100" },
            { source: "trozos_182", target: "trozos_91" },
            { source: "trozos_91", target: "trozos_100" },
            { source: "trozos_91", target: "trozos_66" },
            { source: "trozos_100", target: "trozos_173" },
            { source: "trozos_100", target: "trozos_82" },
            { source: "trozos_66", target: "trozos_62" },
            { source: "trozos_82", target: "trozos_46" },
            { source: "trozos_132", target: "trozos_62" },
            { source: "trozos_62", target: "trozos_151" },
            { source: "trozos_151", target: "trozos_211" },
            { source: "trozos_46", target: "trozos_210" },
            { source: "trozos_90", target: "trozos_210" } // Conexión larga estilo rama exterior
        ]
    };

    // 2. Configuración de dimensiones
    const width = window.innerWidth;
    const height = window.innerHeight;

    const svg = d3.select("#network-svg")
        .attr("width", width)
        .attr("height", height);

    const htmlContainer = d3.select("#html-nodes");

    // 3. Simulación de Fuerzas Física (Estilo Estructural/Árbol)
    const simulation = d3.forceSimulation(dataset.nodes)
        .force("link", d3.forceLink(dataset.links).id(d => d.id).distance(110)) // Distancia entre
nodos
        .force("charge", d3.forceManyBody().strength(-400)) // Repulsión para que parezcan
ramas abiertas
        .force("center", d3.forceCenter(width / 2, height / 2)) // Centrar el árbol en pantalla
        .force("collide", d3.forceCollide().radius(70)); // Evita que las imágenes se encimen
bruscamente

    // 4. Renderizado de las Líneas de Conexión (SVG)
    const link = svg.append("g")
```

```
    .selectAll("line")
    .data(dataset.links)
    .enter().append("line")
    .attr("class", "link");

// 5. Renderizado de los bloques de información (HTML sobrepuesto)
const node = htmlContainer.selectAll(".node-block")
    .data(dataset.nodes)
    .enter().append("div")
    .attr("class", "node-block")
    .call(d3.drag() // Habilitar la interacción para arrastrar las ramas
        .on("start", dragstarted)
        .on("drag", dragged)
        .on("end", dragended));

// Añadir el punto/nodo indicador
node.append("div")
    .attr("class", d => `node-dot ${d.type === 'red' ? 'red' : ''}`);

// Añadir el texto de etiqueta monoespaciado
node.append("span")
    .attr("class", "node-label")
    .text(d => d.label);

// Añadir el fragmento de imagen
node.append("img")
    .attr("class", "node-img")
    .attr("src", d => d.img);

// 6. Actualización en tiempo real (Tick) del motor de física
simulation.on("tick", () => {
    // Actualizar posiciones de las líneas
    link
        .attr("x1", d => d.source.x)
        .attr("y1", d => d.source.y)
        .attr("x2", d => d.target.x)
        .attr("y2", d => d.target.y);

    // Actualizar posiciones de los bloques de imágenes
    node
        .style("left", d => `${d.x}px`)
        .style("top", d => `${d.y}px`);
});

// 7. Funciones de interacción por Arrastre (Drag)
function dragstarted(event, d) {
    if (!event.active) simulation.alphaTarget(0.3).restart();
    d.fx = d.x;
```

```
            d.fy = d.y;
        }

        function dragged(event, d) {
            d.fx = event.x;
            d.fy = event.y;
        }

        function dragended(event, d) {
            if (!event.active) simulation.alphaTarget(0);
            d.fx = null;
            d.fy = null;
        }

        // Redimensionar automáticamente si cambia la ventana
        window.addEventListener('resize', () => {
            const w = window.innerWidth;
            const h = window.innerHeight;
            svg.attr("width", w).attr("height", h);
            simulation.force("center", d3.forceCenter(w / 2, h / 2)).alpha(0.3).restart();
        });
    </script>
</body>
</html>
```

make and xcel to JSON